

Certified cost forecasts and refutable regime maps for exact quantum compilation

Sze Chun Yiu

Stockholm University

A numerical compilation forecast may be supported by a theorem, agreement on tested instances, or a structural model that happens to cover those instances. These warrants are not interchangeable. We separate them for an exact shared-tag Pauli block-encoding model and then test whether the resulting regime description transfers to another compiler family. An interpretable three-family formula first matches a prospectively staged panel, yet a frozen fresh instance refutes it: the formula returns 11 while the exact optimum is 10. A support-two normalization theorem yields a stronger forecaster that equals the unrestricted optimum for every admitted size under the unit structural objective; all finite implementation comparisons agree. Explanation remains separately falsifiable: a repaired borrow vocabulary fails on 64 hostile cases, and six of seven of its complete-census fibres contain mixed exact costs. A later per-block-permutation argument closes the previously open commuting-support-two sector, while one alternative global proof link remains explicitly open, and a second route is refuted by an exact counterexample. In weighted Clifford state preparation, a four-feature separator is minimal on the complete small-size domain, but the frozen next-size rule makes 32 errors on 120 states and shows no improvement under the specified shuffle null ($p = 0.51$). Exact cost, explanation, finite verification and transfer therefore require distinct certificates. The results concern frozen algebraic objectives and do not establish runtime, production-compiler, hardware or quantum-advantage claims.

Sze Chun Yiu: sze-chun.yiu@fysik.su.se

Keywords: quantum compilation; exact optimization; resource forecasting; stabilizer circuits; algorithm selection; verified computation.

1 Introduction

Quantum resource estimates are shaped by compilation choices. Recent systems make hardware assumptions configurable, optimize across full stacks, or predict compilation options from circuit features [1–4]. These developments improve the usefulness of estimates, but they leave a logically prior question: why should a particular numerical prediction be believed? A formula can be correct because a theorem forces it, because a finite benchmark happened to cover the input, or because a structural regime model assigned the right label. The same number can carry any of these warrants.

Verified optimizers establish transformation correctness, while exact synthesis and pattern matching can establish bounded optimality or locally exact rewrites [5–8]. Algorithm selection and instance-space analysis instead ask which method works in which part of a feature space [9–11]. Neither tradition licenses collapsing theorem authority, finite agreement, structural explanation and transfer into one accuracy score. That collapse is especially risky when an interpretable formula survives extensive testing before one exact counterexample defeats it.

We study this separation in an exact shared-tag Pauli block-encoding model derived from Tag-and-Restore Encoding (TARE) [12]. The model admits unrestricted dynamic-programming optima on finite instances and an all-size support-two normalization theorem under a frozen structural objective. This combination makes it possible to distinguish four questions cleanly. Is the predicted cost exact? Does the implementation realize the proved restricted family? Does a com-

compact structural vocabulary explain which minimizing configuration is used? Does the same representation transfer to a materially different compiler family?

The evidence forms one refutation–repair–transfer argument. A compact three-family forecaster first succeeds prospectively, then fails exactly on a fresh three-qubit instance. Replacing that formula with the theorem-backed support-two optimum repairs the cost layer without deleting the counterexample. Hostile tests then refute two successive explanation vocabularies while leaving the cost theorem intact. A complete per-block-permutation analysis closes the previously open commuting-support-two sector, but it does not convert two failed alternative proof chains into successes. Finally, weighted Clifford state preparation shows that even exact finite feature determination need not transfer to the next system size.

The resulting contribution is a layered certificate discipline supported by model-specific theorems and adverse tests. It is not a claim that regime mapping, algorithm selection, Pauli compilation, verified optimization or quantum resource estimation begins here. Nor are the compiler models treated as a statistical sample from which prevalence can be inferred. Their role is discriminating: each exposes a concrete inference that would be valid only if different evidence types were mistakenly conflated.

2 Formal setting and study design

2.1 Exact cost and certificate layers

Let x be an instance, $\mathcal{C}(x)$ its feasible compiler configurations, and $C(c; x, \theta)$ a fixed objective indexed by weights θ . The exact cost is

$$C^*(x; \theta) = \min_{c \in \mathcal{C}(x)} C(c; x, \theta). \quad (1)$$

We distinguish four evidential objects. A *cost certificate* proves a reported value equals Eq. (1). An *implementation check* compares executable code with exact referees or independently checked witnesses. An *explanation certificate* identifies a structural minimizing family or feature regime. A *row-level verification record* states whether a particular emitted row has an exact referee receipt. Failure of one object changes another only when a logical dependency is shown.

For a deterministic reference compiler with cost C_D , its exact region is $\mathcal{R}_D(\theta) = \{x : C_D(x; \theta) = C^*(x; \theta)\}$. When configurations carry a support statistic $s(c)$, the intrinsic support number is

$$\kappa(\theta) = \min\{k : \forall x \exists c^* \text{ with } s(c^*) \leq k\}. \quad (2)$$

A proof of support at most k is only an upper bound until a separate obstruction excludes smaller support.

2.2 Shared-tag Pauli model and forecast families

An n -qubit Pauli word modulo global phase is represented by $(x, z) \in \mathbb{F}_2^{2n}$, with symplectic commutation form $\langle (x, z), (x', z') \rangle = x \cdot z' + z \cdot x' \pmod{2}$. The primary model contains three ordered target pairs. Each block selects two nonidentity frame Paulis that anticommute, one shared Tag Pauli supplies the same two distinct syndrome bits to every block, and one frame per block is assigned the lower structural weight. The unit objective charges weighted frame support, Tag support and a three-block restored-target functional. Supplementary Section S1 gives the complete feasible set and objective.

We use reader-facing notation for four nested cost families. C_{ref} is the original reference construction; $C_{\leq 1}$ is the support-one restricted optimum; $C_{\leq 2}$ is the complete support-at-most-two optimum; and C_{DP} is the unrestricted dynamic-programming optimum. In the archived code these correspond to C_{R6L} , C_{D+} , C_{D++} and C_{DP} , respectively. The initial static forecaster is

$$F_1(x) = \min\{C_{\text{ref}}(x), C_{\leq 1}(x), f_B(x)\}, \quad (3)$$

where B is a named central-borrow family. The theorem-backed forecaster is

$$F_2(x) = C_{\leq 2}(x). \quad (4)$$

The enlarged family B' permits the borrow home to lie outside a block's own target support or in the union support. The hybrid family B'' additionally allows the split-and-borrow support-two shapes exposed by hostile search. These names describe feasible configuration families; agreement of their minima on a panel is not itself a completeness proof.

Table 1: Evidence layers in the shared-tag forecasting sequence. Row counts check implementations or finite explanation domains; all-size authority comes only from the stated compositions. Adverse rows remain in the scientific record after repair.

Object	Evidence	Exact outcome	Authority and boundary
Initial formula F_1	102 prospective rows; 9,546-row frozen benchmark	102/102 prospective labels; one fresh $n = 3$ cost error, 11 versus 10	finite forecast refuted; the counterexample is retained
Support-two forecast F_2	all-size normalization; 9,547 implementation comparison events over 9,546 distinct inputs	$C_{\text{DP}} = C_{\leq 2}$; zero compared-row errors	theorem-backed only for the frozen grammar and unit objective
Enlarged borrow B'	complete 740-instance hostile census	64 strict gaps; six of seven fibres have mixed exact costs	named explanation refuted; support-two cost remains exact
Hybrid envelope B''	10,481 finite rows; 378 geometries $\times 2^{24}$ states	zero finite-panel gap; 6,341,787,648-state commuting-sector residue zero	per-block-permutation route proves Eq. (8); alternative chain/global-completeness route remains open

2.3 Regime representation and transfer

For a frozen feature map $\phi(x)$ and binary label $y(x) = 1[x \in \mathcal{R}_D]$, a feature cell contains instances with the same ϕ . The smallest error attainable by any deterministic classifier using only ϕ on a finite domain is

$$E_{\min}(\phi) = \sum_v \min\{n_0(v), n_1(v)\}, \quad (5)$$

where $n_b(v)$ counts label b in cell v . A positive value is a representation-level impossibility result on that domain. A zero value proves only finite determination. Cell and singleton counts reveal whether that determination compresses the domain or approaches memorization.

Weighted Clifford state preparation supplies the transfer test. Allowed gates are H , S , $S^\dagger$ and controlled-NOT with costs 1, 1, 1, 3. Exact costs are Dijkstra distances from $|0 \cdots 0\rangle$ on complete stabilizer-state graphs of sizes 6, 60, 1,080 and 36,720 for $n = 1, 2, 3, 4$. Complete feature analysis uses all 1,146 states with $n \leq 3$; the next-size test uses a frozen 120-state four-qubit panel. Supplementary Sections S5–S6 specify the deterministic reference synthesis, all 127 features, panel generator, folds and shuffle null.

2.4 Prospective and hostile tests

The forecast sequence was ordered before each corresponding referee was opened. A 102-row boundary panel tested the initial structural labels. A separate 9,546-row benchmark admitted exact refutation of Eq. (3). The repaired implementation in Eq. (4) was then compared on 9,547 comparison rows over the same 9,546 distinct inputs, the retained refuting instance being audited once inside the frozen fresh panel and once as a standalone row. A 740-instance hostile census tested B' without changing its definition; a further 10,481-instance panel tested B'' . The previously open commuting-support-two sector was checked over 378 irreducible geometries and all 2^{24} target-letter states per geometry. The state-preparation features and transfer rule were fixed before four-qubit evaluation. Later repairs never replace earlier failures in the reported denominator.

3 Results

3.1 A support-two theorem makes exact static forecasting possible

Theorem 1 (Support-two normalization). *For every admitted size and target in the shared-tag Pauli model under the unit structural objective,*

Table 2: Cross-model controls for certificate separation. The models are discriminating cases, not independent draws from a compiler population.

Model	Exact authority	Headline result	Boundary retained
Shared-tag Pauli block encoding	all admitted sizes	theorem-backed exact support-two forecast and exact hybrid envelope	objective/grammar specific; an outside-cone support-three control refutes objective independence; two alternative proof links remain open or refuted
Rank-two dependent-triple model	all admitted sizes inside the certificate region	intrinsic support one; syndrome-rank ceiling five	outside-region necessity and a third-family rank-support law are unproved
Six-term linear-combination model	all admitted batches and sizes	exact pair-gain condition for reference optimality	frozen cost model only
Weighted Clifford state preparation	complete $n \leq 3$ graphs; frozen 120-state $n = 4$ panel	exact finite separator, adverse prospective transfer	no production, next-size or universal feature-law claim

Table 3: Feature determination and transfer for weighted Clifford state preparation. The complete-domain separator is exact but the frozen next-size transfer criterion fails.

Quantity	Complete $n \leq 3$	Frozen $n = 4$ panel
States	1,146	120
Full feature cells	1,109	119
Singleton cells	1,072	118
Minimum exact features	4	not established
Parity-split lookup errors	–	32
Covered states	–	2
Shuffle-null mean errors	–	32.41
Empirical p (errors no larger)	–	0.51

an exact optimum exists in which every frame Pauli has support at most two. Consequently,

$$C_{DP}(x) = C_{\leq 2}(x) = F_2(x) \quad (6)$$

for every admitted instance x .

The proof assigns each active qubit of a frame a two-bit class recording local commutation with its partner and with the shared Tag. Global anticommutation makes the first-bit parity odd.

Any class multiset of size at least three contains a removable zero class or equal pair; deleting the corresponding one or two frame letters preserves both parities. A complete 18,432-case local check bounds the restoration increase by the refunded frame cost. Repeated deletion therefore terminates at support two without increasing the objective. Supplementary Section S2 states the arbitrary-size composition and the support-two obstruction to this particular descent.

This theorem is the authority for Eq. (4). The implementation does not call unrestricted dynamic programming in its forecast path. Agreement with a finite referee checks the implementation and its bindings; it is not used to manufacture the theorem.

The same local inequality proves support two throughout the sufficient objective cone $t_c \geq 2t_r$ and $t_{nc} \geq 2t_r$; the unit objective lies on its central boundary. An exact outside-cone control at $(t_c, t_{nc}, t_{tag}, t_r) = (1, 7, 4, 3)$ has a support-three optimum. This refutes an objective-independent support-two claim, but it does not prove that every point outside the cone requires support three or that the global boundary is sharp.

3.2 Prospective success is followed by an exact refutation

The initial formula in Eq. (3) correctly classifies all 102 prospectively staged rows: 90 reference-exact library rows and 12 engineered boundary rows spanning reference, split and borrow cases. The larger frozen benchmark then supplies the necessary hostile opportunity. The formula agrees on all 9,261 structured two-qubit rows and the committed chemistry rows but fails on one fresh seeded three-qubit instance. Its predicted value is 11, whereas unrestricted optimization and a checked support-two witness give 10.

The failure is structural. The witness uses a support-two frame whose effective borrow home lies outside that block’s own target support. Enlarging the borrow family to B' repairs this row. More importantly, Eq. (6) yields F_2 , which returns 10 and agrees with exact cost on all 9,547 comparison events, which cover the same 9,546 distinct inputs: the retained refuting row is audited once inside the frozen fresh panel and once as a standalone re-audit row. The original error remains part of the result: the cost layer was repaired by moving to a proved family, not by redefining the failed forecast.

3.3 Exact cost survives two explanation attacks

The first hostile explanation panel finds 64 instances among 740 for which

$$C_{\leq 2} < \min\{C_{\leq 1}, f_{B'}\}. \quad (7)$$

Each gap is one unit, and a representative witness combines a weight-two Tag with a phantom borrow. The same rows continue to satisfy Eq. (6); they refute the named explanation, not the cost.

The complete 740-instance census also removes a misleading regularity produced by selecting only Eq. (7). Across all rows the exact offset distribution relative to $f_{B'}$ is $\{0 : 389, 1 : 325, 2 : 13, 3 : 13\}$. Six of seven $f_{B'}$ fibres contain more than one exact cost. Restricting to the 64 selected gaps yields the apparently uniform offset one and no mixed fibre. Thus the uniformity is a selection effect, while the full census proves that $f_{B'}$ alone cannot be an exact explanation on this domain.

Adding the hybrid family B'' yields zero gap on a separate 10,481-instance panel. A later

all-size argument closes the previously open commuting-support-two sector by allowing independent target permutations within each remaining block. For each of 378 irreducible geometries, all 2^{24} target-letter states admit a non-increasing alternative with fewer commuting support-two blocks. The complete domain therefore contains 6,341,787,648 exact states and zero residue. Composed with the previously proved reductions in this route, it gives

$$C_{DP} = \min\{C_{\leq 1}, f_{B'}, f_{B''}\} \quad (8)$$

under the unit objective.

Two negative proof results remain essential to the interpretation. A separate single-pinner normalization is valid over 133,349,376 composite configurations, but its composition into a global chain is unproved. Its machine fields therefore remain `CHAIN-ALL-N = false` and `GLOBAL-B''-COMPLETENESS = false`. A proposed two-coordinate chain representation is also refuted by an exact three-Tag configuration. These failures do not contradict Eq. (8), whose proof uses per-block target permutations; neither may be cited as an independent proof of it.

3.4 Cross-model controls separate proof ceilings from intrinsic support

Two additional exact models test whether the certificate distinctions are merely terminological. In a rank-two dependent-triple Pauli model, a conserved-syndrome deletion abstraction gives a safe support ceiling of five, whereas a whole-system Tag reconstruction proves intrinsic support one: support one always suffices and support zero is infeasible. The support-one proof holds inside an explicit objective-weight region; outside that region the certificate is silent. A corrected search over 211,248 frozen-domain candidates finds no strict outside witness under four objectives, which is a bounded negative rather than global sharpness.

In a six-term linear-combination model, the reference construction is exact if and only if a proved pair-gain condition holds for every admitted batch and size. This supplies the opposite control: a low-order exact boundary can exist even when minimizing witnesses use larger block structure. The complete regression covers 729 instances at $n = 1$ and 38,760 at $n = 2$ with zero

mismatches. These two results are used only to test the certificate schema; they do not enlarge Eq. (6) or Eq. (8) to other grammars.

3.5 Finite feature determination fails to transfer

For weighted Clifford state preparation, exact shortest-path censuses find 5, 28 and 189 reference-exact states among the complete 6, 60 and 1,080 states for $n = 1, 2, 3$. Strict improvements fall into four mechanism classes: processing order, pivot choice, elimination route and global synthesis family. A prospectively frozen next-size forecast then fails, matching 100 of 120 regime labels and 67 of 120 exact costs.

The original 13-feature representation contains 12 mixed cells and forces at least 43 errors among the 1,146 complete small-size states. An enlarged 127-feature vocabulary removes all mixed cells, producing 1,109 cells including 1,072 singletons. Exhaustive search over all one-, two- and three-feature subsets finds no exact separator; the four features consisting of first schedule cost, maximum factor cost, the sum of squared per-qubit H counts and maximum per-qubit S count form an exact separator with 523 cells. Dropping any one feature restores a positive error floor. The minimum separator size is therefore exactly four on $n \leq 3$. It contains no sign-aware state coordinate, so the proposed state-sign mechanism is not supported.

The compact separator still does not license transfer. Under the frozen parity-split lookup protocol on 120 four-qubit states, the richer representation covers only two states and makes 32 errors. The specified 200-permutation shuffle null has mean 32.41 errors, and 51% of permutations have an error count no larger than 32. The result shows no improvement under this null on this panel; it is neither an equivalence test nor a claim that no transferable representation exists.

4 Discussion

The shared-tag sequence establishes a distinction that a final zero-error table would conceal. The first forecast was interpretable and prospectively successful, yet wrong. The second is exact because a support theorem constrains every admitted instance, not because 9,547 compari-

son events over 9,546 distinct inputs happened to agree. Explanation then fails twice without changing that cost. A certified forecast is therefore not one status: its value, implementation, explanation and row-level referee state have different failure conditions.

The commuting-support-two result also illustrates why failed proof routes must remain visible. The successful per-block-permutation argument proves the exact envelope in Eq. (8). The single-pinner calculation proves a strong local statement but lacks a global chain, while the two-coordinate representation is refuted. Treating either local success as the global theorem would overstate the evidence; suppressing them after another route succeeds would erase useful information about which proof abstractions fail.

This separation clarifies the relation to algorithm selection. A performance footprint answers which method works on which features. An exact regime record can additionally ask whether a restricted family is sufficient for every optimum, whether the bound is tight, whether a compact explanation is identifiable, and what happens outside a proof-validity region. The state-preparation result shows why these coordinates cannot be filled by analogy. Four features determine a complete small-size domain, yet the rule does not survive the frozen next-size test.

For a compilation or verification reader, the same record can be read as a study in warrant typing for static cost forecasts. Three warrant types appear. A *theorem warrant* — here the support-two normalization and the per-block-permutation envelope — licenses an all-size statement for the frozen grammar and objective; its cost of obtaining is a proof plus complete-domain machine checks, and it says nothing about other grammars or objectives. An *agreement warrant* — finite comparison against the unrestricted dynamic program — licenses only an implementation check on the compared rows; it is the cheapest to obtain and regenerate, and it transfers to no untested size, which is exactly what the staged panel’s 102/102 agreement followed by a single fresh-instance refutation demonstrates. A *structural-coverage warrant* — a named explanation family covering all current minimizing witnesses — licenses a search restriction rather than a value; obtaining it costs a hostile census, and here it was refuted twice without disturbing the

cost certificate. What the separation offers to this reader is a measurement of relative certification cost for static forecast cost: agreement is cheapest and weakest, structural coverage is intermediate and refutable, and theorem warrants are expensive but are the only ones that transfer across system sizes. Whether a theorem warrant is cheaply available for a given compiler depends on whether its grammar admits a normal-form or domination argument; this paper exhibits one grammar where it was obtainable and delimits what it then covers.

The result is also orthogonal to improving the physical fidelity of resource estimation. Compilation-driven and full-stack estimators aim to replace coarse fixed assumptions with configurable hardware/software models [1, 2]. Description-aware objectives separate execution cost from the complexity of specifying a circuit [13]. The present question is different: whatever quantity is estimated, what certifies the reported value and what independently certifies its explanation? A more detailed cost model still benefits from keeping theorem, finite validation, structural interpretation and transfer distinct.

For compiler studies the practical reporting order follows. State the model and objective before quoting exactness. Identify the unrestricted reference and the restricted forecast family. Preserve every exact counterexample in the denominator. Report whether a normal form is merely sufficient or also tight. Test explanation vocabularies against complete fibres rather than only selected gaps. When a feature law is found, disclose its cell structure and freeze a transfer coordinate. These steps do not guarantee publication or practical usefulness, but they prevent several common evidential promotions.

5 Related work and contribution boundary

Algorithm selection and instance-space analysis already establish the general programme of mapping feature-conditioned performance regions [9–11]. Quantum work has used instance spaces for parameter initialization, selected between quantum and classical methods, and predicted compilation options or target devices [3, 4, 14, 15]. The regime-map language in this paper is an evidential organization of exact results, not a claim to

have invented feature-conditioned compiler selection.

Exact, verified and algebraic optimization is likewise established. Prior work supplies verified circuit rewrites and oracle compilation, exact pattern matching, graph-theoretic simplification and bounded optimal Clifford synthesis [5–8, 16]. Benchmark suites characterize compiler behavior across circuits and devices [17, 18]. These works absorb any claim that formal checking, exact search, compiler benchmarking or Clifford synthesis is introduced here.

The Pauli model descends from TARE and established Pauli compilation and partitioning [12, 19–22]. Recent Hamiltonian-simulation compilation further emphasizes global binary-symplectic simplification [23]. The structural objectives studied here do not replace hardware-aware resource estimation, connectivity analysis or production compilation.

The closest residual is the combination demonstrated by the refutation chain: an all-size exact restricted-family cost, a separately falsifiable explanation family, prospectively staged and hostile exact-referee tests, complete-fibre audits, and row-specific verification authority. The cross-model controls show why these fields are not redundant. Conversely, this paper does not provide a new production optimizer, a broad benchmark, external hardware validation, or a universal compiler theory. Its empirical and theorem claims stop at the declared algebraic models and finite transfer panel.

6 Limitations

The all-size claims concern frozen algebraic grammars and structural support objectives. Structural support is not gate count, circuit depth, latency, qubit count, fault-tolerant overhead, error rate or quantum advantage. Other TARE ranks, objectives outside the proved support-two cone, routing constraints and hardware cost vectors require new proofs. The exact envelope in Eq. (8) is therefore not a production resource estimator.

The implementation evidence is extensive but dependent. Thousands of rows are generated within a small number of formal families and do not constitute thousands of independent compiler domains. The 6.34-billion-state calculation closes a finite local proof obligation whose arbitrary-

size consequence comes from the analytic composition. Computational scale improves checking; it does not by itself establish novelty, practical prevalence or external validity.

The state-preparation conclusions are finite. Complete graphs are used only through $n = 3$, and transfer is tested on one frozen 120-state $n = 4$ panel. The four-feature separator is exact on the smaller domain, but its mechanism attribution fails and its transfer result is adverse. Other representations, gate sets, costs, folds or measurement-assisted synthesis may behave differently.

The remaining proof-route failures are not missing optional polish. The single-pinner chain and its global B'' completeness implication remain unproved, while the two-coordinate chain premise is refuted. Eq. (8) relies on a different per-block-permutation route. All checks were performed within the same research programme and are not external proof certification or independent replication. All gold labels, audits, and adjudications in this programme are same-programme and single-author work using AI assistance; no result reported here has been checked by an out-of-programme reviewer or an independent implementation outside this repository.

7 Conclusion

A static compiler forecast can be exact without having a simple exact explanation, and a complete finite explanation can fail to transfer. In the shared-tag model, a prospectively successful compact formula is refuted by one exact fresh instance. A support-two theorem repairs the cost layer for every admitted size, while later hostile censuses refute the first repaired explanation. A different per-block-permutation proof then closes the remaining commuting-support-two sector without laundering the failed alternative proof routes.

Weighted Clifford state preparation supplies the cross-model boundary within the formal programme: four features separate every complete $n \leq 3$ state, yet the frozen $n = 4$ test remains adverse and the proposed sign mechanism is unsupported. The strongest supported conclusion is therefore a certificate separation—exact cost, executable implementation, structural explanation, row-level verification and transfer are dif-

ferent scientific objects. Keeping them separate makes negative results informative and prevents finite accuracy from being mistaken for proof or generality.

8 Data and code availability

The submission package contains the exact result records, frozen protocols, selected analysis code, complete state-preparation panel records, table-generation code and independent or separately implemented checkers used for the headline values. A machine-readable manifest binds every included file and both manuscript PDFs. The public repository is <https://github.com/SzeChunYiu/ORION>; a permanent archive DOI and final reuse licence are not asserted in this version. Row-level exactness is reported only where an exact referee or theorem binding exists.

Ethics, resources and author statement

This study used no human participants, animals, clinical records or personal data. Computation supported exact enumeration, shortest-path optimization and finite proof obligations. Computational scale is a verification cost, not evidence of practical quantum advantage.

Sze Chun Yiu is the sole named author and retains responsibility for the scientific claims, analyses and final manuscript. Generative artificial-intelligence tools assisted with editorial refinement, literature-discovery support, code assistance and simulated review; all scientific statements and submission materials remain subject to author verification.

References

- [1] Colin Campbell, Rich Rines, Victory Omole, Tina Oberoi, Palash Goiporia, Rayat Roy, R. Peyton Cline, Eric B. Jones, and Teague Tomesh. “Resource estimation via efficient compilation of key quantum primitives” (2026). [arXiv:2604.01376](https://arxiv.org/abs/2604.01376).
- [2] Harshkumar Oza, Aritra Sarkar, Syed Naqi Abbas, Rahul Bhowmick, Aryan Prakash, Prateek P. Kulkarni, and Krishna Kumar Sabapathy. “AutoQuREO: A framework for

- automated quantum resource estimation and optimization” (2026). [arXiv:2608.12936](#).
- [3] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. “Predicting good quantum circuit compilation options”. In 2023 IEEE International Conference on Quantum Software. **Pages 43–53**. (2023).
 - [4] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. “MQT Predictor: Automatic device selection with device-specific circuit compilation for quantum computing”. **ACM Transactions on Quantum Computing 6**, 1–26 (2025).
 - [5] Kesha Hietala, Robert Rand, Shih-Han Hung, Xiaodi Wu, and Michael Hicks. “A verified optimizer for quantum circuits”. **Proceedings of the ACM on Programming Languages 5**, 1–29 (2021).
 - [6] Liyi Li, Finn Voichick, Kesha Hietala, Yuxiang Peng, Xiaodi Wu, and Michael Hicks. “Verified compilation of quantum oracles”. **Proceedings of the ACM on Programming Languages 6**, 589–615 (2022).
 - [7] Raban Iten, Romain Moyard, Tony Metger, David Sutter, and Stefan Woerner. “Exact and practical pattern matching for quantum circuit optimization”. **ACM Transactions on Quantum Computing 3**, 1–41 (2022).
 - [8] Sergey Bravyi, Joseph A. Latone, and Dmitri Maslov. “6-qubit optimal Clifford circuits”. **npj Quantum Information 8**, 79 (2022).
 - [9] John R. Rice. “The algorithm selection problem”. In *Advances in Computers*. **Volume 15**, pages 65–118. Elsevier (1976).
 - [10] Kate A. Smith-Miles. “Cross-disciplinary perspectives on meta-learning for algorithm selection”. **ACM Computing Surveys 41**, 1–25 (2009).
 - [11] Kate Smith-Miles and Mario Andrés Muñoz. “Instance space analysis for algorithm testing: Methodology and software tools”. **ACM Computing Surveys 55**, 1–31 (2023).
 - [12] Niclas Schillo, Andreas Sturm, and Rüdiger Quay. “TARE: Block encoding linear combinations of pauli strings without ancilla state preparation” (2026). [arXiv:2601.05740](#).
 - [13] Aritra Sarkar. “DeComp2: Description complexity aware decomposition” (2026). [arXiv:2607.23990](#).
 - [14] Vivek Katal, Kate Smith-Miles, Charles Hill, and Lloyd Hollenberg. “On the instance dependence of parameter initialization for the quantum approximate optimization algorithm: Insights via instance space analysis”. **INFORMS Journal on Computing 37**, 146–171 (2025).
 - [15] Charles Moussa, Henri Calandra, and Vedran Dunjko. “To quantum or not to quantum: towards algorithm selection in near-term quantum optimization”. **Quantum Science and Technology 5**, 044009 (2020).
 - [16] Ross Duncan, Aleks Kissinger, Simon Perdrix, and John van de Wetering. “Graph-theoretic simplification of quantum circuits with the ZX-calculus”. **Quantum 4**, 279 (2020).
 - [17] Ang Li, Samuel Stein, Sriram Krishnamoorthy, and James Ang. “QASMBench: A low-level quantum benchmark suite for NISQ evaluation and simulation”. **ACM Transactions on Quantum Computing 4**, 1–26 (2023).
 - [18] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. “MQT Bench: Benchmarking software and design automation tools for quantum computing”. **Quantum 7**, 1062 (2023).
 - [19] Artur F. Izmaylov, Tzu-Ching Yen, Robert A. Lang, and Vladyslav Verteletskyi. “Unitary partitioning approach to the measurement problem in the variational quantum eigensolver method”. **Journal of Chemical Theory and Computation 16**, 190–195 (2020).
 - [20] Filipa C. R. Peres and Ernesto F. Galvão. “Quantum circuit compilation and hybrid computation using pauli-based computation”. **Quantum 7**, 1126 (2023).
 - [21] Ewout van den Berg and Kristan Temme. “Circuit optimization of hamiltonian simulation by simultaneous diagonalization of pauli clusters”. **Quantum 4**, 322 (2020).
 - [22] Jennifer Paykin, Albert T. Schmitz, Mohannad Ibrahim, Xin-Chuan Wu, and A. Y. Matsuura. “PCOAST: A pauli-based quantum circuit optimization framework”. In 2023 IEEE International Conference on Quantum Computing and Engineering. **Pages 715–726**. (2023).
 - [23] Zhaohui Yang, Yuwei Han, Ruiyun Zhang, Dawei Ding, Jianxin Chen, Yuan Feng,

and Yuan Xie. “Efficient compilation for hamiltonian simulation via global binary symplectic form simplification” (2026). [arXiv:2608.11579](#).